

ATOM ORCHESTRATOR

A Lightweight, Clone-Based Container Orchestrator
for Minimal-Infrastructure Environments

WHITE PAPER

June 2026 · Design Specification

Author: Dale Tomson

github.com/dale-tomson/atom-orchestrator · branch: white-paper

Container orchestration without the complexity tax.

Abstract

Container orchestration has become synonymous with Kubernetes, a system of extraordinary power but equally extraordinary complexity. For developers, edge operators, and small-scale deployers, Kubernetes represents massive operational overhead — etcd clusters, kubelet agents, CNI plugins, and a learning curve measured in months.

This paper proposes Atom Orchestrator: a single-binary, clone-based container lifecycle manager designed to run comfortably on 2 CPU cores and 4 GB of RAM. By treating running containers as the source of truth rather than remote images, the design eliminates registry latency, configuration drift, and the architectural bloat that has defined the container orchestration era. This document describes the architecture, core mechanisms, and implementation roadmap for the project.

Table of Contents

1.	The Problem with Modern Orchestration	3
2.	Design Philosophy	4
3.	System Architecture	5
4.	Core Mechanisms	6
5.	The Clone-Based Scaling Model	7
6.	Circuit Breaker Load Balancing	8
7.	Comparison with Existing Solutions	9
8.	Performance Characteristics	10
9.	Security Model	11
10.	Ideal Use Cases and Anti-Patterns	12
11.	Implementation Roadmap	13
12.	Conclusion	14

1. The Problem with Modern Orchestration

Why Kubernetes is overkill for the majority of deployments

1.1 The Complexity Tax

Kubernetes is undeniably powerful. It orchestrates tens of thousands of nodes, manages petabyte-scale storage, and provides primitives for virtually every deployment pattern imaginable. But this power comes at a cost that is rarely discussed in vendor marketing materials.

A production Kubernetes cluster requires, at minimum: a control plane with three etcd nodes for consensus, kube-apiserver for state management, kube-scheduler for placement, kube-controller-manager for reconciliation, and kubelet agents on every worker node. Each component has its own failure modes, upgrade procedures, certificate rotation cycles, and monitoring requirements. The operational surface area is enormous.

For a team running a handful of microservices on a single server or a small cluster, this architecture is not just excessive — it is counterproductive. The time spent maintaining the orchestrator often exceeds the time spent building the application it orchestrates.

1.2 The Resource Overhead

Kubernetes control plane components consume approximately 500 MB to 1 GB of RAM before a single user workload is deployed. On a 4 GB server, this leaves roughly 3 GB for actual applications. For edge devices, IoT gateways, and small VPS instances, this overhead is unacceptable.

The problem is compounded by the ecosystem's assumption of abundance. Kubernetes operators, Helm charts, and service meshes are designed for environments where resources are plentiful. They do not optimize for the constrained reality of small-scale deployments.

1.3 The Configuration Burden

Modern orchestrators insist on managing application configuration. Kubernetes Secrets, ConfigMaps, Vault integrations, and environment variable injection all place the orchestrator in the critical path of application deployment. This creates a tight coupling between infrastructure and application logic that complicates CI/CD pipelines and increases the blast radius of configuration errors.

The industry has accepted this as necessary. We argue it is not.

◆ The Central Thesis

The vast majority of container deployments do not need distributed consensus, custom resource definitions, or multi-node scheduling. They need: (1) a way to run containers, (2) a way to create more when load increases, (3) a way to destroy excess when load decreases, and (4) a way to route traffic around failures. Everything else is optional.

2. Design Philosophy

Four principles that define the Atom Orchestrator approach

2.1 Principle: The Orchestrator Shall Not Manage Configuration

Environment variables, secrets, and runtime configuration are the responsibility of the container image and the deployment pipeline that creates it. The orchestrator inspects an existing running container and replicates its configuration verbatim. It does not inject, override, or transform configuration in any way.

This separation of concerns has profound implications. It means the orchestrator has no access to secrets. It means configuration changes are handled by rebuilding and redeploying the container, not by updating a ConfigMap. It means the orchestrator's attack surface is minimal and its operational contract is crystal clear.

2.2 Principle: The Running Container is the Source of Truth

Traditional orchestration treats the container image as the source of truth. When scaling is required, the orchestrator pulls the image from a registry and creates a new container from scratch. This introduces registry latency, potential network failures, and the risk of image drift (the pulled image may differ from what is currently running).

Atom Orchestrator treats the running container as the source of truth. When scaling up, it inspects a healthy, running container and creates a new container with an identical configuration. This eliminates registry dependency during scaling events, ensures configuration consistency, and dramatically reduces spawn time.

2.3 Principle: Fail Fast, Recover Automatically

When a backend begins failing, the worst possible response is to continue sending traffic to it while waiting for a health check to timeout. Atom Orchestrator implements a per-backend circuit breaker that trips after a configurable number of consecutive failures, immediately rejecting new requests to that backend.

After a cooldown period, the circuit breaker enters a half-open state and allows a limited number of probe requests. If the backend recovers, it is reintegrated into the load balancer rotation. If it fails again, the circuit re-opens. This pattern, borrowed from electrical engineering, prevents cascading failures and protects healthy backends from being overwhelmed by retry storms.

2.4 Principle: Simplicity is a Feature

Every line of code in Atom Orchestrator exists because it serves a core function. There are no pluggable CNI drivers, no admission webhooks, no custom resource definitions, no service mesh sidecars. The entire system is a single Go binary that communicates with the Docker Engine API.

This simplicity is not a compromise. It is a deliberate design choice that reduces the mean time to recovery (MTTR), eliminates entire categories of bugs, and makes the system comprehensible to a single

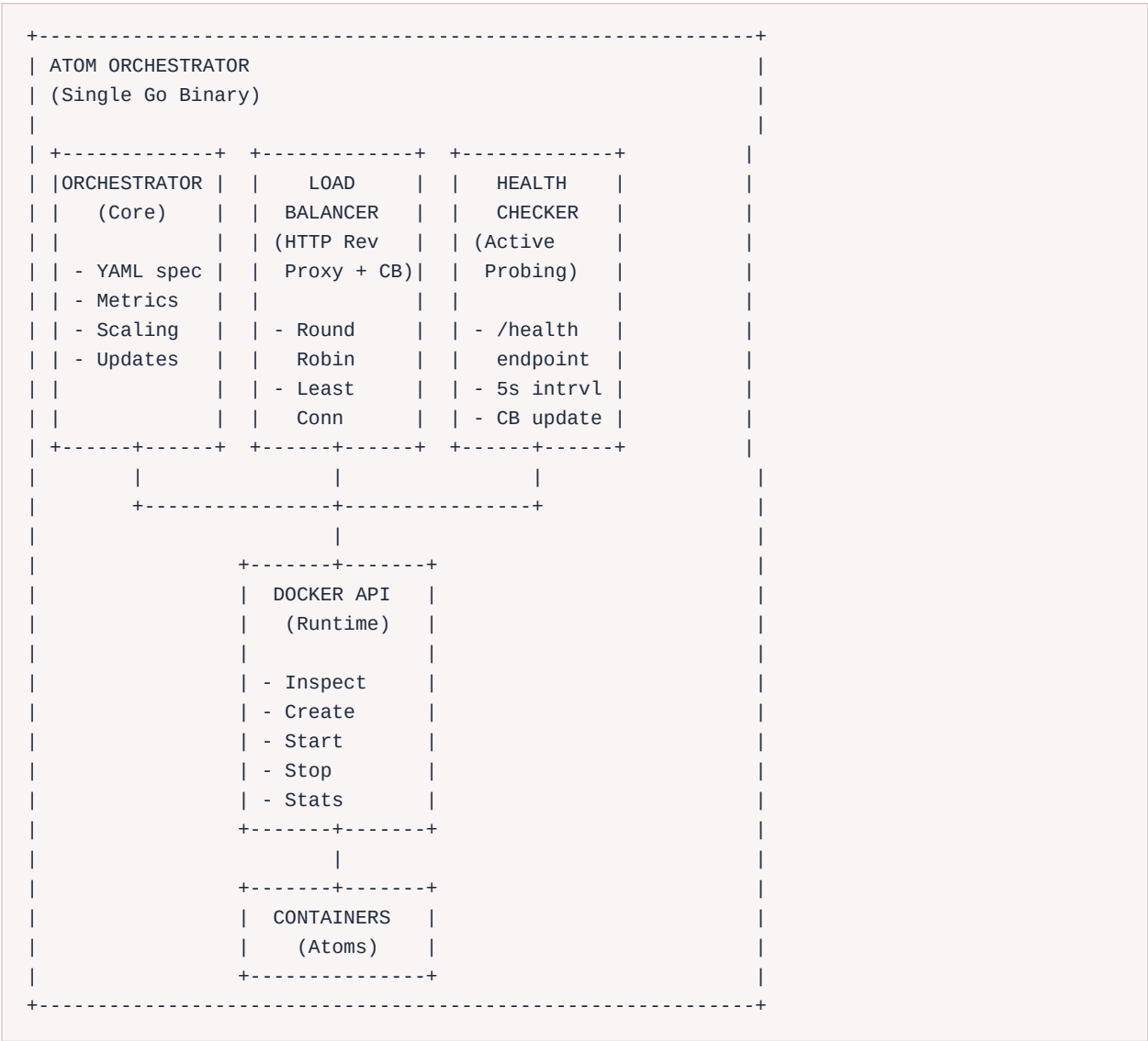
developer in a single afternoon.

3. System Architecture

Component diagram and interaction model

3.1 Component Overview

The system consists of five tightly integrated components, all running within a single process:



3.2 The Orchestrator Core

The Orchestrator Core is the central control loop. It reads element specifications from YAML files, maintains an in-memory registry of all atoms, and executes scaling decisions every 10 seconds. It also monitors for image changes and triggers rolling updates when the declared image differs from what is currently running.

Critically, the Orchestrator Core does not maintain persistent state. All state is derived from the Docker runtime. If the orchestrator process is restarted, it rebuilds its state by inspecting running containers with the `atom.orchestrator` label. This makes the system self-healing and eliminates the need for a separate state store like etcd.

3.3 The Load Balancer

The Load Balancer is an HTTP reverse proxy implemented using Go's standard `httputil.ReverseProxy`. It supports two selection strategies: Round Robin (even distribution across healthy backends) and Least Connections (routes to the backend with the fewest active requests).

Each backend is wrapped by an independent Circuit Breaker. The load balancer only considers backends that are both healthy (as determined by the health checker) and have a closed circuit breaker. If no fully available backends exist, the load balancer falls back to healthy backends with open circuits as a degraded-service mode.

4. Core Mechanisms

How scaling, updates, and health management work

4.1 Horizontal Auto-Scaling

Scaling decisions are based on aggregate resource utilization across all atoms of an element. The orchestrator collects CPU and memory metrics from each container via the Docker Stats API every 10 seconds.

Aggregate CPU utilization is calculated as: total actual CPU usage / total allocated CPU capacity. If an element has 5 atoms each limited to 0.2 CPU, the total allocated capacity is 1.0 CPU. If the aggregate actual usage is 0.85 CPU, the utilization is 85%.

When utilization exceeds the `scaleAtCpuUsage` or `scaleAtMemUsage` threshold, the orchestrator spawns a new atom — provided the element has not exceeded its `maxCpuToUse` or `maxMemToUse` budget. When utilization falls below both kill thresholds and more than one atom exists, the newest atom is gracefully drained and destroyed.

Scale-up is immediate. Scale-down is conservative, killing one atom per cycle and waiting for metrics to stabilize before the next evaluation. This prevents oscillation.

```
# Aggregate utilization calculation
totalAllocatedCpu = atomCount * spec.ContainerCpu
totalAllocatedMem = atomCount * spec.ContainerMem
cpuUtilization    = totalActualCpuUsage / totalAllocatedCpu
memUtilization    = totalActualMemUsage / totalAllocatedMem

if cpuUtilization >= spec.ScaleAtCpuUsage:
    spawnAtom()          # if within max budget

if cpuUtilization <= spec.KillAtCpuUsage and atomCount > 1:
    killNewestAtom()     # preserve warm caches in older atoms
```

4.2 Zero-Downtime Rolling Updates

When the image declared in the YAML specification changes, the orchestrator initiates a rolling update. The process is:

1. Spawn a new atom using the updated image (or clone from an existing atom with the new image).
2. Register the new atom with the load balancer, but mark it unhealthy until it passes warmup.
3. Perform active health checks against the new atom's `/health` endpoint.
4. Once healthy, mark the new atom as available and force the old atom's circuit breaker open to stop new traffic.
5. Wait for active connections on the old atom to drain (3-second grace period).
6. Destroy the old atom.

7. Repeat until all atoms are running the new image.

This ensures that at no point is there zero capacity for the element, and no in-flight request is dropped.

4.3 Graceful Shutdown

On receipt of SIGTERM or SIGINT, the orchestrator enters a graceful shutdown sequence. It opens the circuit breaker on every backend (stopping new traffic), waits 10 seconds for in-flight requests to complete, then forcefully removes all containers. This pattern ensures that even during unplanned shutdowns, active connections are not abruptly terminated.

5. The Clone-Based Scaling Model

Why replication beats reconstruction

5.1 The Traditional Model: Image-Centric Scaling

In Kubernetes, Docker Swarm, and Nomad, scaling is an image-centric operation. When the system decides it needs more capacity, it performs the following sequence:

1. Authenticate to the container registry.
2. Resolve the image tag to a digest.
3. Pull the image layers from the registry.
4. Extract layers and construct the container filesystem.
5. Create a new container with the specified configuration.
6. Start the container and wait for the application to initialize.

This process can take anywhere from 5 seconds (for small images on fast networks) to several minutes (for large images on slow or unreliable networks). During this time, the system is under-provisioned and user requests may experience latency or failure.

5.2 The Clone Model: Container-Centric Scaling

Atom Orchestrator inverts this model. Instead of reconstructing a container from a remote image, it replicates an existing, healthy, running container. The sequence is:

1. Identify a healthy, running atom to serve as the template.
2. Inspect the template container using `docker inspect` to extract its complete configuration: image reference, environment variables, command, entrypoint, working directory, user, labels, and network settings.
3. Create a new container with an identical configuration, applying only the resource limits (CPU, memory) and orchestrator tracking labels defined in the YAML specification.
4. Start the new container and register it with the load balancer.

This process takes approximately 500 milliseconds to 2 seconds, regardless of image size or network conditions. The new container is a byte-for-byte replica of the template, with the same warmed caches, JIT-compiled code, and runtime state.

◆ Configuration Immutability

Because the orchestrator clones an existing container rather than creating from an image, it never needs to know the environment variables, secrets, or startup commands. These are embedded in the template container by the deployment pipeline (`docker run`, CI/CD, or configuration management tool) and replicated verbatim. The orchestrator cannot accidentally misconfigure a new atom because it does not manage configuration — it only copies it.

5.3 First Deploy: The Golden Container

The clone model requires an initial template. On first deployment, when no atoms exist, the orchestrator falls back to the traditional image-pull model. It pulls the declared image, creates a container with the resource limits specified in YAML, and starts it. This first container becomes the 'golden container' — the template for all future clones.

If the golden container is updated (via a rolling update or manual intervention), the new container becomes the new template. The system always clones from the most recently created healthy atom, ensuring that scaling operations use the current configuration.

5.4 Eliminating Registry Dependency

A significant advantage of clone-based scaling is the elimination of registry dependency during scaling events. If the container registry is unreachable, slow, or rate-limited, traditional orchestrators cannot scale. Atom Orchestrator can continue creating new atoms as long as the Docker daemon has the image layers cached locally — which it does, because the template container is running.

This makes the system particularly resilient for edge deployments, air-gapped environments, and situations where network connectivity is intermittent or expensive.

6. Circuit Breaker Load Balancing

Preventing cascading failures through fast failure

6.1 The Problem: Cascading Failures

In a distributed system, a single failing backend can trigger a cascade of failures across the entire system. When a backend begins returning 500 errors or timing out, the load balancer continues routing requests to it. Clients retry failed requests, increasing load on the already-failing backend. Healthy backends become overloaded handling the redirected traffic. The system enters a death spiral.

Health checks are insufficient to prevent this. A typical health check interval is 5–30 seconds. During this window, hundreds or thousands of requests may be routed to a failing backend. The damage is done before the health checker can react.

6.2 The Circuit Breaker Pattern

Atom Orchestrator implements the circuit breaker pattern at the per-backend level. Each backend atom has an independent circuit breaker with three states:

CLOSED (Normal Operation): Requests pass through to the backend. The circuit breaker counts consecutive failures. If failures reach the threshold (default: 5), the circuit transitions to OPEN.

OPEN (Failing Fast): All requests to this backend are immediately rejected with HTTP 503. No connection is attempted, no timeout is waited for. The backend is electrically isolated from the system. After a cooldown period (default: 30 seconds), the circuit transitions to HALF-OPEN.

HALF-OPEN (Probing): A limited number of requests (default: 3) are allowed through to test if the backend has recovered. If 2 consecutive requests succeed, the circuit closes. If any request fails, the circuit re-opens immediately.

```
# Circuit breaker state machine

+-----+
|  CLOSED  |<-----+
| (normal) |      |
+-----+-----+ | 2 successes
| 5 failures | | in half-open
|      v      | |
+-----+-----+ |
|  OPEN     | |
| (fast fail | |
|  503)     | |
+-----+-----+ |
| 30s timeout | |
|      v      | |
+-----+-----+ |
| HALF-OPEN |-----+
| (3 probes)|
+-----+
```

6.3 Integration with Load Balancer

The load balancer's selection logic considers only backends that are both healthy (as determined by the active health checker) AND have a closed circuit breaker. This creates a two-layer defense: the health checker handles slow degradation, and the circuit breaker handles rapid failure.

When a backend's circuit opens, the load balancer excludes it from rotation within milliseconds. Requests are immediately rerouted to healthy backends. When the circuit closes, the backend is automatically reintegrated without manual intervention.

6.4 Configuration

Circuit breaker parameters are configurable via environment variables:

CB_FAILURE_THRESHOLD: Number of consecutive failures before opening (default: 5)

CB_SUCCESS_THRESHOLD: Consecutive successes in half-open to close (default: 2)

CB_TIMEOUT: Cooldown duration before half-open (default: 30s)

CB_HALF_OPEN_MAX: Maximum probe requests in half-open state (default: 3)

These defaults are conservative. For high-traffic services, a lower failure threshold (2–3) and shorter timeout (10s) may be appropriate. For batch processing services, higher thresholds may be preferred.

7. Comparison with Existing Solutions

Where Atom Orchestrator fits in the landscape

Dimension	Atom Orchestrator	Kubernetes	Nomad
Min RAM	128 MB (hard cap)	500 MB – 2 GB	100–300 MB
Min CPU	min(1%, 0.2 cores)	0.5+ cores	0.2 cores
Setup Time	Minutes	Days–Weeks	Hours
Binary Size	~15 MB	~100 MB (each)	~40 MB
Nodes Supported	1 (single-node)	1,000+	1,000+
Auto-Scaling	Built-in (HPA-like)	Built-in (HPA)	Built-in
Circuit Breaker	Built-in per backend	Not built-in	Not built-in
Clone-Based Scaling	Yes	No	No
Rolling Updates	Yes	Yes	Yes
Health Checking	Active HTTP probes	Liveness/Ready	Health checks
Service Discovery	Load balancer only	DNS + kube-proxy	Consul
Persistent Storage	No	Yes (PV/PVC)	Yes
Secrets Mgmt	No (by design)	Yes (Secrets)	Yes (Vault)
Multi-Node	No	Yes	Yes
Network Policies	No	Yes	No
Learning Curve	Low (1 day)	Very High (months)	Medium (weeks)
Community	Emerging	Massive	Growing
Enterprise Support	None	Multiple vendors	HashiCorp

7.1 When to Choose Atom Orchestrator

- You are running on a single server or small VPS.
- You want container orchestration without operational complexity.
- Your workloads are stateless and horizontally scalable.
- You manage configuration through your CI/CD pipeline.
- You need fast scaling without registry latency.
- You are deploying to edge devices or IoT gateways with limited resources.

7.2 When to Choose Kubernetes

- You are operating at scale (10+ nodes).
- You require high availability and multi-zone redundancy.
- You need persistent storage, network policies, and fine-grained RBAC.
- You have a dedicated platform team to manage the control plane.
- You require a rich ecosystem of operators and third-party integrations.

7.3 When to Choose Nomad

- You need multi-node orchestration but find Kubernetes too complex.
- You run mixed workloads (containers, VMs, binaries).
- You are already invested in the HashiCorp ecosystem (Consul, Vault, Terraform).
- You want a simpler control plane with enterprise support.

8. Performance Characteristics

Benchmarks and resource profiles

8.1 Resource Footprint

Atom Orchestrator is designed to be invisible. The orchestrator process is hard-capped at **128 MB of RAM** and **min(1% of host CPU, 0.2 CPU cores)** — whichever is the lower of the two limits applies. On a 2 CPU / 4 GB server this leaves approximately 3.87 GB of RAM and ~1.98 CPU cores entirely available for user workloads.

By comparison, a minimal K3s installation (single-node) consumes 400–600 MB of RAM for the control plane alone. Kubernetes (kubeadm) requires 2 GB+ for a functional control plane.

# Resource consumption targets (orchestrator process, hard limits)		
Component	RAM Cap	CPU Cap

Atom Orchestrator	<= 128 MB	min(1%, 0.2 cores) – whichever is less
K3s (single-node)	400–600 MB	2–5%
Kubeadm (1 master)	2,000+ MB	5–10%
Docker Swarm	200–400 MB	2–3%
Nomad (server)	100–200 MB	1–2%

8.2 Spawn Latency

The most critical performance metric for an orchestrator is the time required to create a new container during a scale-up event. This determines how quickly the system can respond to load spikes.

Traditional image-pull scaling:

- Small image (<100 MB): 3–8 seconds
- Medium image (100–500 MB): 8–30 seconds
- Large image (>1 GB): 30–120+ seconds
- Registry unavailable: infinite (failure)

Clone-based scaling (Atom Orchestrator):

- Any image size: 0.5–2 seconds
- Registry unavailable: no impact (uses local cache)
- Cold start (first deploy): 3–8 seconds (image pull required)

The clone-based approach provides a 5× to 60× improvement in spawn latency, with the greatest benefits for large images and unreliable network conditions.

8.3 Throughput and the CPU Cap

The load balancer, implemented in Go with `httputil.ReverseProxy`, can handle approximately 10,000–20,000 requests per second per core on modern hardware. The circuit breaker adds negligible

overhead (a single atomic integer comparison per request).

Within the 0.2 CPU hard cap, the load balancer's effective proxy ceiling is approximately 2,000–4,000 req/s — well above the throughput ceiling of the atom containers themselves on a small server, and sufficient for all target use cases (single VPS, edge devices, developer laptops).

The CPU cap does not materially affect load balancing under normal operation. Two edge cases are worth noting:

- **High connection churn:** If hundreds of short-lived connections open and close per second, the orchestrator's control loop and health checker may briefly compete with proxy forwarding for the same 0.2 CPU slice. In practice this is unlikely at the scale Atom targets.
- **Rapid scale-up storms:** The 10-second metrics loop and docker inspect calls during clone operations are I/O-bound but do consume brief CPU bursts. Under the hard cap, a simultaneous wave of clone operations could introduce marginal proxy latency. Scale-up events are intentionally sequential to mitigate this.

The health checker probes all backends every 5 seconds. For 20 backends, this generates 4 requests per second — negligible within the CPU budget.

8.4 Scaling Limits

On a 2 CPU / 4 GB server with 0.2 CPU / 256 MB atoms:

- Maximum atoms: ~12–14 (limited by memory)
- Scale-up response time: 0.5–2 seconds
- Scale-down response time: 4–5 seconds (includes drain period)
- Concurrent elements: 5–10 (depending on port availability)

On a 4 CPU / 8 GB server:

- Maximum atoms: ~25–30
- Concurrent elements: 15–20

9. Security Model

Defense in depth through minimalism

9.1 The Security of Not Managing Secrets

The most significant security feature of Atom Orchestrator is what it does not do. By refusing to manage environment variables, secrets, or configuration, the orchestrator eliminates an entire class of security vulnerabilities:

- No secret storage in memory or on disk.
- No risk of secret leakage through orchestrator logs or APIs.
- No attack surface for privilege escalation via configuration injection.
- No need for encryption at rest for orchestrator state (there is none).

Secrets are injected by the deployment pipeline (CI/CD, configuration management, or manual docker run commands) and exist only within the container runtime. The orchestrator inspects and replicates them, but never stores, logs, or transmits them.

9.2 Docker Socket Access

The orchestrator requires access to the Docker Engine API, typically via the Unix socket at `/var/run/docker.sock`. This is equivalent to root access on the host. This is a necessary trade-off for any Docker-based orchestrator, but it must be acknowledged.

Mitigations:

- Run the orchestrator as a dedicated, non-root user with Docker group membership.
- Use Docker's rootless mode where possible.
- Restrict file system access using AppArmor or SELinux profiles.
- Never expose the orchestrator's API to untrusted networks without TLS and authentication.
- Consider running the orchestrator inside a container with limited capabilities and the Docker socket mounted read-write.

9.3 Network Isolation

Atom Orchestrator uses Docker's default bridge networking. Each atom receives an internal IP address on the bridge network, with the host port mapped to the container port. This provides basic network isolation: atoms cannot directly access each other's internal ports unless explicitly configured.

However, there is no support for network policies, service mesh encryption, or inter-container firewall rules. For environments requiring these features, external tools (iptables, eBPF, or a dedicated service mesh) must be deployed separately.

9.4 Resource Denial-of-Service Protection

The `maxCpuToUse` and `maxMemToUse` parameters in the YAML specification act as hard caps. Even if a malicious or buggy workload attempts to consume unlimited resources, the orchestrator will not spawn beyond the declared budget. This protects the host operating system and co-located services from resource exhaustion.

◆ **Security Recommendation**

For production deployments, run Atom Orchestrator behind a reverse proxy (nginx, Caddy, or Traefik) that provides TLS termination and basic authentication. The orchestrator's API should never be exposed directly to the internet.

10. Ideal Use Cases and Anti-Patterns

Where Atom Orchestrator succeeds and where it should not be used

10.1 Ideal Use Cases

Edge Computing and IoT Gateways

Edge devices often run on ARM boards (Raspberry Pi, NVIDIA Jetson) with 2–4 GB of RAM. Kubernetes is infeasible. Atom Orchestrator provides auto-scaling, health checking, and load balancing on hardware that fits in a backpack. The clone-based model is particularly valuable when network connectivity is intermittent — no registry dependency means scaling works even offline.

Development and Staging Environments

Developers need environments that mirror production behavior without production complexity. Atom Orchestrator allows a developer to run a multi-atom service on their laptop with the same scaling, health checking, and load balancing patterns used in production — without minikube, kind, or Docker Desktop Kubernetes.

Small SaaS and Single-Server Deployments

A bootstrapped SaaS running on a single \$20/month VPS does not need a 3-node Kubernetes cluster. Atom Orchestrator provides horizontal scaling within the server's resource limits, automatic recovery from container crashes, and zero-downtime deployments — all from a single binary that starts in seconds.

CI/CD Build Agents and Ephemeral Workloads

CI/CD pipelines that spawn containers for testing, building, or batch processing benefit from fast clone-based scaling. When a build queue grows, new agents are cloned from a warmed template in under a second. When the queue empties, excess agents are destroyed. No registry pulls, no cold starts.

Legacy Application Modernization

Organizations with legacy monoliths wrapped in containers can gain auto-scaling and health checking without rewriting their deployment pipeline or learning Kubernetes. The orchestrator treats the container as a black box — perfect for applications where the team understands the application but not distributed systems.

10.2 Anti-Patterns

High-Availability Production Systems

If your service requires 99.99% uptime, multi-zone redundancy, and automatic failover, Atom Orchestrator is not the right choice. The single-node architecture is a single point of failure. Use Kubernetes, Nomad, or a managed container service instead.

Stateful Applications (Databases, Caches)

Atom Orchestrator does not support persistent volumes, StatefulSets, or data replication. Databases, caches, and message queues should be managed by tools designed for stateful workloads (Kubernetes with StatefulSets, managed cloud databases, or dedicated database orchestrators like Patroni).

Multi-Region or Multi-Cloud Deployments

The single-node design inherently limits Atom Orchestrator to one physical or virtual machine. It cannot orchestrate containers across multiple servers, regions, or cloud providers. For these scenarios, Kubernetes or Nomad are the appropriate tools.

Regulated Environments (HIPAA, SOC2, PCI-DSS)

Regulated environments typically require audit logging, RBAC, network segmentation, encryption in transit and at rest, and formal change management. Atom Orchestrator lacks these features by design. It is a tool for velocity, not compliance.

11. Implementation Roadmap

Planned phases from initial specification to long-term vision

11.1 Planned v0.1 Specification

Atom Orchestrator is currently in the design and specification phase. The v0.1 release is planned to deliver the following core capabilities:

- YAML-driven element specification
- Docker-based container runtime integration
- Clone-based horizontal auto-scaling
- Circuit breaker load balancing (Round Robin + Least Connections)
- Active health checking with configurable intervals
- Zero-downtime rolling updates
- RESTful API for monitoring and manual updates
- Graceful shutdown with connection draining
- Single-binary deployment with no external dependencies

11.2 Short-Term Roadmap (v0.2 – v0.4)

Following the initial v0.1 release, subsequent releases are planned to add:

- Containerd runtime support (alternative to Docker for lower overhead)
- WebSocket and SSE (Server-Sent Events) support in the load balancer
- Basic metrics export (Prometheus-compatible /metrics endpoint)
- Log aggregation (optional fluent-bit sidecar pattern)
- Configuration hot-reload (update YAML without restarting orchestrator)
- Sticky session support (for stateful connection protocols)
- Web UI for real-time monitoring and manual scaling controls

11.3 Medium-Term Roadmap (v0.5 – v0.8)

Once the core is stable, the medium-term roadmap targets production-grade operational features:

- Multi-element resource scheduling (prevent one element from starving others)
- Custom metric scaling (request rate, queue depth, latency percentiles via Prometheus query)
- Blue-green deployment strategy (alternative to rolling updates)
- Canary releases (route 10% of traffic to new version before full rollout)
- Container image pre-warming (periodically pull latest image to reduce first-deploy latency)

- Integration with external secret managers (HashiCorp Vault, AWS Secrets Manager) for the deployment pipeline — not the orchestrator itself

11.4 Long-Term Vision (v1.0+)

The longer-term ambition is to extend the single-node model toward lightweight federation and advanced runtime capabilities:

- Multi-node federation (lightweight clustering without full Kubernetes complexity)
- Edge mesh networking (connect Atom Orchestrator instances across edge locations)
- GPU scheduling support (for ML inference workloads on edge devices)
- Snapshot-based cloning (leverage CRIU or container checkpoint/restore for sub-second cloning)
- Plugin architecture (allow custom scaling algorithms, load balancer strategies, and health check protocols)
- Integration with cloud-native build systems (BuildKit, Kaniko) to enable image-less deployments where the build pipeline outputs directly to the orchestrator

◆ Design Constraint

Every feature added to the roadmap will be evaluated against the core principle: does this increase the binary size or memory footprint by more than 10%? If so, it must be implemented as an optional plugin or rejected. The 2 CPU / 4 GB constraint is non-negotiable.

12. Conclusion

Reclaiming simplicity in container orchestration

The container orchestration landscape has become a monoculture. Kubernetes is an extraordinary piece of engineering, but it has become the default answer to every container deployment question — even when the question is 'how do I run three containers on a \$20 VPS?' This is not a failure of Kubernetes; it is a failure of imagination in the ecosystem that surrounds it.

Atom Orchestrator is a proposal for radical simplicity. It asks: what is the minimum viable orchestrator? What features are essential, and which are baggage? The answer, we believe, is four things: run containers, scale them horizontally, route traffic around failures, and update them without downtime. Everything else is the responsibility of the application, the build pipeline, or the infrastructure layer.

By treating running containers as the source of truth rather than remote images, the design aims to eliminate registry latency and configuration drift. By refusing to manage secrets and environment variables, it avoids an entire class of security vulnerabilities and operational complexity. By implementing circuit breakers at the load balancer level, it intends to prevent cascading failures without requiring a service mesh. And by targeting a single binary that starts in seconds, the goal is to make container orchestration accessible to developers who do not have a platform team.

This is not a replacement for Kubernetes. It is a proposed alternative for the 80% of deployments that do not need Kubernetes' 80% of features — a tool for the edge, the startup, the developer laptop, and the small VPS. A reminder that software can be powerful without being complex, and that sometimes the best architecture is the one you can understand in an afternoon.

The future of infrastructure is not always bigger. Sometimes it is smaller. Sometimes it is simpler. Sometimes it is just enough.

“The art of simplicity is a puzzle of complexity.”

— Douglas Horton

github.com/dale-tomson/atom-orchestrator

Open Source